PEPTIDE QSAR PREDICTION TOOL: FULL SCIENTIFIC AND FUNCTIONAL DOCUMENTATION

Project name: Peptide QSAR Prediction Tool
Document purpose: manuscript-ready software and methods description
Target users: peptide scientists, bioinformaticians, medicinal chemists, and Windows users without programming experience
Current implementation: local Streamlit application written in Python
Developer profile: Ahmed G. Soliman, MEXT master's student at Kyutech, School of Life Science and Engineering; biotechnology background from Ain Shams University. Developer profile: https://sites.google.com/view/ahmed-g-soliman/home

---

1. SCIENTIFIC PURPOSE

The Peptide QSAR Prediction Tool is a peptide-focused quantitative structure-activity/property relationship platform. It converts peptide primary sequences into numerical descriptors that capture composition, physicochemical behavior, sequence patterns, and peptide-like structural tendencies. These descriptors are used to train supervised machine learning models for regression, binary classification, or multiclass classification.

The scientific assumption is that peptide activity or peptide physicochemical behavior can be partially explained by measurable sequence-derived properties, including amino acid composition, charge, hydrophobicity, molecular size, aromaticity, residue-class balance, aliphatic content, stability-related indices, and local sequence patterns. The application is therefore designed for peptide QSAR rather than small-molecule QSAR. It does not require SMILES as the primary input, although SDF files can be parsed when they contain an explicit peptide sequence property.

Typical biological or physicochemical targets include:

- Experimental activity values such as IC50, MIC, EC50, inhibition percentage, toxicity, hemolysis, antimicrobial potency, or binding score.
- Docking-derived targets such as docking score, Vina affinity, Glide score, binding energy, or any numeric docking/property column supplied by the user.
- Classification labels such as active/inactive, toxic/non-toxic, soluble/insoluble, or multiple peptide activity classes.

---

2. SOFTWARE ARCHITECTURE

The application is organized as a modular Python project. The graphical user interface is implemented with Streamlit because it is stable on Windows, easy to launch through a `.bat` file, and suitable for non-programmers. Core scientific calculations are performed by Python

packages and custom sequence-based implementations.

2.1 Main Files

File	Role
`app.py`	Main Streamlit interface and tab routing.
`modules/io_utils.py`	Input parsing, FASTA/text/table/SDF loading, sequence validation.
`modules/descriptor_engine.py`	Peptide descriptor calculation.
`modules/preprocessing.py`	Missing value handling, scaling, feature filtering, splitting, PCA/embedding.
`modules/modeling.py`	Model catalog, model training, prediction, persistence, feature importance, SHAP support.
`modules/evaluation.py`	Regression and classification metrics.
`modules/visualization.py`	Interactive Plotly visualizations.
`modules/reporting.py`	CSV/Excel/HTML export utilities.
`modules/structure_criteria.py`	Practical peptide design-quality criteria.
`modules/design_suggestions.py`	Position-level residue-effect analysis and mutation suggestions.
`modules/constants.py`	Amino acid scales, residue classes, motifs, and atom composition constants.
`data/example_peptides.csv`	Example dataset for testing and demonstration.
`peptide_qsar_single_file.py`	Single-file launcher source for easier PyInstaller packaging.
`run_app.bat`	Windows launcher that creates/uses a virtual environment and opens the app.

---

3. USER WORKFLOW

The tool follows a complete peptide QSAR workflow:

1. User enters or uploads peptide sequences.
2. The app validates amino acid sequences and reports invalid characters.
3. Peptide descriptors are calculated from sequence.
4. Descriptor tables are inspected and exported.
5. User chooses a target column and task type.
6. Multiple machine learning models are trained and compared.
7. The best selected model is evaluated by metrics and diagnostic plots.
8. New peptide sequences are predicted with the trained or loaded model.
9. New peptides are ranked and evaluated with design-quality criteria.
10. Results, tables, model files, and reports are exported.

---

4. INPUT MODULE

4.1 Accepted Input Types

The application accepts:

- Manual single sequence input, for example `AKLVFF`.
- Manual multi-sequence pasted input.
- FASTA text or FASTA files.
- CSV files.
- Excel files.
- TXT files.
- SDF files when the SDF contains a peptide sequence property such as `Sequence`, `Peptide Sequence`, or `AA Sequence`.

4.2 Why SDF Requires a Sequence Field

SDF is primarily a chemical structure table format. A peptide sequence cannot always be reconstructed reliably from atom coordinates or connectivity alone, especially for modified residues, missing hydrogens, salts, non-standard residues, or docking output files. For scientific reproducibility, this tool treats SDF as a property container and extracts peptide sequence only when a sequence field is present or when a valid amino-acid-like value can be detected. Docking scores or binding energies can be read as target columns if included as numeric properties.

4.3 Sequence Validation

The input module standardizes sequences by:

- Removing whitespace and formatting characters.
- Converting to uppercase.
- Keeping only standard one-letter amino acid codes.
- Flagging invalid or non-standard characters.
- Detecting duplicate sequences.

This validation is important because most implemented descriptors, including ProtParam-derived molecular weight, pI, charge, instability, and GRAVY, assume standard amino acid residues.

---

5. DESCRIPTOR CALCULATION MODULE

The descriptor engine converts each peptide sequence into a numeric feature vector. These descriptors are biologically meaningful for peptides because peptide activity often depends on amphipathicity, cationicity, hydrophobicity, residue composition, size, and local residue patterns.

5.1 Core Physicochemical Descriptors

Descriptor	Scientific meaning	Relevance to peptides
`Length`	Number of residues.	Influences molecular size, flexibility, proteolytic susceptibility, membrane penetration, and synthesis feasibility.
`MolecularWeight`	Approximate molecular mass from amino acid composition.	Important for pharmacokinetics, diffusion, permeability, and experimental characterization.
`Aromaticity`	Frequency of aromatic residues.	Aromatic residues can enhance membrane interaction, binding, and pi-related interactions.
`Hydrophobicity_KD` / `Gravy`	Average Kyte-Doolittle hydropathy score.	Hydrophobicity strongly affects membrane affinity, solubility, aggregation, and binding.
`NetCharge_pH7`	Estimated net peptide charge at pH 7.0.	Cationic charge is central for many antimicrobial and cell-penetrating peptides.
`IsoelectricPoint`	pH at which net charge is approximately zero.	Helps characterize charge state under experimental pH conditions.
`InstabilityIndex`	Empirical stability index from sequence dipeptide statistics.	Used as an approximate indicator of protein/peptide stability.
`AliphaticIndex`	Relative volume of aliphatic side chains A, V, I, L.	Often associated with thermostability and hydrophobic core-like character.
`BomanIndex`	Average protein-binding potential-like scale.	Used as an approximate peptide interaction propensity indicator.
`ShannonEntropy`	Diversity of amino acid composition.	Low entropy may indicate repetitive or biased sequences; high entropy indicates diverse residue composition.
`ChargeDensity`	Net charge divided by peptide length.	Captures whether charge is concentrated in short peptides or diluted in longer peptides.
`HydrophobicMoment100`	Approximate helical hydrophobic moment using 100 degrees per residue.	Estimates amphipathic alpha-helical character, relevant to membrane-active peptides.

5.2 Amino Acid Composition (AAC)

AAC descriptors measure the fraction of each of the 20 standard amino acids:

`AAC_A`, `AAC_C`, `AAC_D`, ..., `AAC_Y`

Scientific basis:

- AAC is a common protein/peptide representation that captures residue enrichment.
- It is sequence-order-independent, so it is robust for small datasets.
- It can reveal whether activity correlates with cationic residues (K/R/H), hydrophobic residues (A/V/I/L/F/W/Y/M), acidic residues (D/E), or special residues such as P, G, and C.

5.3 Dipeptide Composition (DPC)

When enabled, DPC computes the relative frequency of all 400 adjacent residue pairs:

`DPC_AA`, `DPC_AC`, ..., `DPC_YY`

Scientific basis:

- DPC captures local sequence-order information that AAC misses.
- Many peptide activities depend on neighboring residues, local motifs, amphipathic arrangement, and short sequence patterns.
- DPC is powerful but can increase feature count substantially, so it should be used carefully with small datasets.

5.4 Residue Class Frequencies

Residues are grouped into biologically meaningful classes:

- Hydrophobic.
- Polar.
- Positive.
- Negative.
- Aromatic.
- Aliphatic.
- Sulfur-containing.
- Tiny/small residues.
- Helix-breaking/flexible residues where relevant.

Scientific basis:

- Peptide function often depends on residue class balance rather than only exact residue identity.
- For example, antimicrobial peptides often combine positive charge with hydrophobic or amphipathic residue patterns.
- Solubility, toxicity, aggregation, and binding can be affected by these broad residue classes.

5.5 Elemental Composition

Elemental descriptors estimate counts and fractions for:

- Carbon.
- Hydrogen.
- Nitrogen.
- Oxygen.
- Sulfur.

Scientific basis:

- Elemental composition relates to molecular formula-like information.
- Sulfur is useful for cysteine-containing peptides.
- Nitrogen/oxygen content may correlate with polarity, hydrogen bonding, and solubility.

5.6 Motif Fingerprints

Motif fingerprints count selected short sequence patterns normalized by sequence length. The motif list is defined in `modules/constants.py` and can be modified by the user.

Scientific basis:

- Some peptide families are enriched in specific motifs or residue pairs.
- Motif counts provide interpretable features for known design rules.
- Motif fingerprints are intentionally simpler than molecular fingerprints and are sequence-based for peptides.

5.7 Secondary-Structure Propensity Descriptors

The tool includes:

- `HelixFraction`
- `TurnFraction`
- `SheetFraction`
- `HelixPropensityAvg`
- `SheetPropensityAvg`

Scientific basis:

- Peptide activity may depend on the tendency to form helices, turns, or extended conformations.
- Chou-Fasman-type propensities are empirical sequence-derived indicators, not full 3D predictions.
- These descriptors are best interpreted as approximate tendencies, especially for short peptides.

5.8 Important Descriptor Limitations

The tool calculates descriptors from primary sequence. It does not directly simulate:

- Full 3D peptide folding.
- Solvent-specific conformational ensembles.
- Post-translational modifications unless encoded manually.
- Non-natural amino acids unless additional constants and validation rules are added.
- Protonation microstates beyond approximate net charge calculations.

Therefore, descriptors should be interpreted as QSAR features, not as direct experimental measurements.

---

6. DATA PREPROCESSING MODULE

The preprocessing module prepares descriptors for machine learning.

6.1 Missing Value Handling

Numeric missing values are handled by imputation. This prevents model training from failing when a descriptor has missing or undefined values.

Scientific reason:

- Experimental peptide datasets are often small and incomplete.
- Imputation allows supervised learning while preserving samples, but excessive missingness should be reported.

6.2 Feature Scaling

Supported scaling methods include:

- Standard scaling.
- Robust scaling.
- Min-max scaling.
- No scaling.

Scientific reason:

- Linear models, SVM/SVR, kNN, and neural networks are sensitive to feature magnitude.
- Tree-based models are less scale-sensitive, but consistent preprocessing is still useful for model comparison.

6.3 Variance Threshold

Low-variance features can be removed because they provide little discriminating information.

Scientific reason:

- If every peptide has almost the same descriptor value, that descriptor cannot explain activity differences.

6.4 Correlation Filtering

Highly correlated descriptors can be filtered.

Scientific reason:

- Many peptide descriptors are mathematically related, such as length and molecular weight.
- Reducing collinearity can improve interpretability and reduce overfitting.

6.5 PCA Analysis

Principal Component Analysis projects descriptor matrices into orthogonal components.

Scientific reason:

- PCA helps visualize descriptor-space clustering.
- PCA loadings show which descriptors drive the main variation.
- PCA is unsupervised; it explains descriptor variance, not necessarily biological activity.

---

7. MACHINE LEARNING AND QSAR MODELING

The modeling module supports regression, binary classification, and multiclass classification.

7.1 Regression Models

Regression models predict continuous targets such as IC50, MIC, docking score, solubility score, toxicity percentage, or binding energy.

Implemented models include:

- Linear Regression.
- Ridge Regression.
- Lasso Regression.

- Elastic Net.
- Random Forest Regressor.
- Support Vector Regression with RBF kernel.
- Gradient Boosting Regressor.
- Extra Trees Regressor.
- k-nearest neighbors regressor.
- MLP Regressor.
- Optional XGBoost Regressor.
- Optional LightGBM Regressor.

7.2 Classification Models

Classification models predict labels such as active/inactive, toxic/non-toxic, soluble/insoluble, or multi-class peptide categories.

Implemented models include:

- Logistic Regression.
- Random Forest Classifier.
- SVM with RBF kernel.
- Gradient Boosting Classifier.
- Extra Trees Classifier.
- k-nearest neighbors classifier.
- Gaussian Naive Bayes.
- MLP Classifier.
- Optional XGBoost Classifier.
- Optional LightGBM Classifier.

7.3 Why Include Nonlinear Models?

Peptide activity frequently depends on nonlinear interactions between descriptors. For example:

- High positive charge may help membrane binding only within a suitable hydrophobicity range.
- Too much hydrophobicity can improve membrane activity but also increase toxicity or aggregation.
- Dipeptide patterns may interact with charge and length.

For this reason, the tool includes nonlinear models such as SVR/SVM, kNN, Random Forest, Extra Trees, Gradient Boosting, XGBoost, LightGBM, and MLP.

7.4 Cross-Validation

Cross-validation is used to estimate generalization performance. The tool uses:

- KFold for regression.
- StratifiedKFold for classification when class counts allow it.

Scientific reason:

- Training and evaluating on the same data can produce overly optimistic results.
- Cross-validation is especially important for small peptide datasets.

7.5 Negative R2 Warning

For regression, R2 can be negative when the model predicts worse than a simple mean-target baseline on the test set.

Scientific interpretation:

- Negative R2 is not a software error.
- It usually indicates weak descriptor-target relationship, too little data, noisy measurements, overfitting, poor train/test split, or target values that require transformation such as log(IC50).

---

8. EVALUATION MODULE

8.1 Regression Metrics

Metric	Meaning
`R2`	Fraction of variance explained relative to a mean baseline.
`RMSE`	Root mean squared prediction error; penalizes large errors.
`MAE`	Mean absolute prediction error; easier to interpret in target units.

8.2 Classification Metrics

Metric	Meaning
Accuracy	Fraction of correct predictions.
Precision	Fraction of predicted positives that are true positives.
Recall	Fraction of true positives recovered by the model.
F1 score	Harmonic mean of precision and recall.
ROC-AUC	Binary discrimination ability when probabilities are available.
Confusion matrix	Counts of correct and incorrect predictions by class.

8.3 Scientific Interpretation

No single metric is sufficient. For peptide QSAR:

- Regression should report R2, RMSE, MAE, dataset size, and split method.
- Classification should report class balance and F1 score, not only accuracy.
- For imbalanced active/inactive peptide datasets, F1 and recall may be more informative than accuracy.

---

9. PREDICTION MODULE

The prediction module:

1. Receives new peptide sequences.
2. Validates sequences.
3. Calculates the same descriptor set used during training.
4. Applies the saved preprocessing pipeline.
5. Predicts activity/property/class using the trained model.
6. Ranks peptides by predicted score.
7. Exports prediction results.

Scientific reason:

- Prediction must use the same descriptor calculation and preprocessing learned during training.
- Saving the model together with preprocessing prevents accidental mismatch between training and prediction features.

---

10. STRUCTURE-QUALITY CRITERIA FOR NEW PEPTIDES

The tool includes a practical peptide design-quality scoring layer. This is not a replacement for wet-lab validation, molecular dynamics, or toxicity assays. It is a transparent heuristic screen to help users identify peptides with more favorable sequence-derived profiles.

Criteria include:

- Length suitability.
- Net charge range.
- Hydrophobicity/GRAVY range.
- Charge density.
- Aromaticity.
- Aliphatic index.

- Instability index.
- Boman index.
- Hydrophobic moment.

Scientific basis:

- Peptides with extreme hydrophobicity may aggregate or become toxic.
- Very low charge may reduce interaction with negatively charged membranes for antimicrobial/cell-penetrating use cases.
- Very high positive charge may increase nonspecific interactions.
- Instability and aliphatic indices provide approximate stability-related signals.
- Amphipathic tendency can be important for membrane-active peptides.

The criteria are intentionally adjustable because the optimal property window depends on the target application.

---

11. POSITION-LEVEL DESIGN SUGGESTIONS

The design-suggestion module analyzes residue-position relationships when aligned or similar-length peptides are available.

11.1 Method

For each sequence position and residue:

1. The tool creates a binary variable indicating whether a peptide contains that residue at that position.
2. The binary variable is compared with the activity/property target.
3. Pearson correlation and mean target shift are calculated.
4. A design score is assigned according to the optimization direction.
5. Candidate residue substitutions are suggested for a selected peptide.

11.2 Optimization Direction

The tool infers whether higher or lower target values are preferred from the target column name:

- Lower-is-better examples: IC50, MIC, EC50, Ki, Kd, docking score, binding energy, Vina score.
- Higher-is-better examples: activity, inhibition, solubility, score, potency.

The user should verify this direction because experimental naming conventions vary.

11.3 Scientific Interpretation

Position-level suggestions are hypothesis-generating only. They can identify associations such as:

- A lysine at position 3 is associated with higher activity.
- A phenylalanine at position 5 is associated with lower docking energy.
- A glycine at a certain position appears unfavorable in the current dataset.

However, these are not causal proof. Suggested mutations should be filtered through chemical feasibility, known peptide biology, docking, synthesis constraints, and experimental validation.

---

12. EXPLAINABILITY

The application supports:

- Model-native feature importance for tree-based models.
- Coefficients for linear models when available.
- Permutation-like or fallback summaries where appropriate.
- Optional SHAP importance if the SHAP package is installed and compatible with the trained estimator.

Scientific reason:

- QSAR models should not only predict but also help interpret which descriptors influence predictions.
- Feature importance can reveal whether activity is driven by charge, hydrophobicity, residue composition, length, motifs, or other descriptors.

Important limitation:

- Feature importance reflects the fitted model and dataset. It does not prove a descriptor is mechanistically causal.

---

13. VISUALIZATION MODULE

The app provides interactive Plotly visualizations:

Plot	Scientific use
Descriptor distribution	Detect descriptor ranges, skew, and outliers.

Correlation heatmap	Identify redundant or strongly related descriptors.
PCA scatter	Visualize descriptor-space clustering.
PCA explained variance	Determine how much variance is captured by early PCs.
PCA loadings	Identify descriptors driving PC axes.
Model comparison chart	Compare trained models using the selected metric.
Feature importance chart	Interpret influential descriptors.
Prediction ranking plot	Rank candidate peptides by predicted activity/property.
Actual vs predicted plot	Diagnose regression calibration and bias.
Residual plot	Detect systematic regression errors.
Confusion matrix heatmap	Diagnose classification errors between classes.
Structure criteria radar/distribution	Visualize new-peptide design-quality profiles.
Position-residue heatmap	Identify position-specific residue associations.

---

14. REPORT EXPORT

The reporting module exports:

- CSV tables.
- Excel workbooks.
- HTML reports.
- Trained model files in `.joblib` format.
- PDF documentation included in the project.

The report can include:

- Input summary.
- Descriptor summary.
- Selected model.
- Evaluation metrics.
- Prediction table.
- Model comparison.
- Timestamp.

---

15. FUNCTION-LEVEL DOCUMENTATION

This section summarizes the main functions currently implemented in the project.

15.1 `app.py`

| Function | Purpose |

| --- | --- |
`_init_session_state`	Initializes Streamlit session variables for input, descriptors, trained models, predictions, and UI state.
`_sanitize_filename`	Converts model/report names into safe filenames for Windows.
`_load_example_df`	Loads the included example peptide dataset.
`_model_file_list`	Lists saved `.joblib` models.
`_clear_downstream_after_new_input`	Clears descriptors/models/predictions after new input to avoid stale results.
`_plot`	Displays Plotly charts with unique Streamlit keys to prevent duplicate element errors.
`_render_header`	Renders the application title and visual header.
`_render_home_tab`	Shows overview, workflow, and beginner guidance.
`_render_upload_tab`	Handles manual input and file upload.
`_render_descriptors_tab`	Calculates and displays peptide descriptors.
`_render_train_tab`	Selects target/task/model options and trains models.
`_render_evaluate_tab`	Shows metrics, diagnostic plots, and model summaries.
`_resolve_bundle_for_prediction`	Chooses the active trained or loaded model for prediction.
`_render_predict_tab`	Predicts new peptides and ranks candidates.
`_render_visualization_tab`	Displays descriptor, PCA, model, prediction, structure, and design plots.
`_render_export_tab`	Exports CSV/Excel/HTML reports and model files.
`_render_about_tab`	Shows documentation, scientific basis, references, and developer information.
`main`	Configures Streamlit and routes the nine dashboard tabs.

15.2 `modules/io_utils.py`

Function/Class	Purpose
`ValidationResult`	Dataclass storing valid rows, invalid rows, duplicate rows, and warnings.
`clean_sequence`	Normalizes user sequence text.
`_normalize_header`	Standardizes column/property names for robust parsing.
`parse_fasta_text`	Parses FASTA-formatted sequences into a table.
`parse_sequence_block`	Parses plain pasted sequence blocks.
`parse_sdf_text`	Parses SDF records and extracts peptide sequence plus properties.
`_decode_bytes`	Decodes uploaded bytes into text.
`guess_sequence_column`	Finds likely sequence column names.
`guess_target_columns`	Finds likely activity/property target columns.
`read_uploaded_table`	Reads CSV, Excel, TXT, FASTA, and SDF uploads into a DataFrame.
`validate_sequences`	Validates sequence characters and duplicate sequences.
`merge_manual_and_uploaded`	Combines manual and uploaded data.

| `to_fasta` | Exports sequence records to FASTA text. |

15.3 `modules/descriptor_engine.py`

Function/Class	Purpose
`DescriptorConfig`	Controls optional descriptor groups such as DPC, motifs, and elemental descriptors.
`_safe_protein_analysis`	Creates a Biopython `ProteinAnalysis` object.
`_aa_composition`	Computes amino acid composition.
`_dipeptide_composition`	Computes adjacent residue-pair composition.
`_sequence_entropy`	Computes Shannon entropy of residue composition.
`_aliphatic_index`	Computes Ikai-style aliphatic index.
`_boman_index`	Computes average Boman-like binding-potential scale.
`_hydrophobic_moment`	Estimates alpha-helical hydrophobic moment.
`_average_propensity`	Averages residue propensity scales.
`_residue_class_frequencies`	Computes residue-class frequencies.
`_elemental_composition`	Estimates elemental counts and fractions.
`_motif_fingerprints`	Computes motif frequencies.
`calculate_sequence_descriptors`	Calculates all descriptors for one peptide.
`calculate_descriptors_dataframe`	Calculates descriptors for all peptides in a table.
`get_descriptor_columns`	Identifies descriptor columns in a DataFrame.
`describe_descriptor_set`	Returns a human-readable descriptor-set summary.

15.4 `modules/preprocessing.py`

Function/Class	Purpose
`SplitConfig`	Stores train/test split settings.
`PreprocessingConfig`	Stores imputation, scaling, variance, and correlation filtering settings.
`_get_scaler`	Selects the requested scaler.
`FeaturePreprocessor`	Fits and applies preprocessing transformations.
`split_dataset`	Splits descriptor data into train/test sets.
`compute_embedding`	Computes PCA/t-SNE/UMAP-style embedding when available.
`compute_pca_analysis`	Computes PCA scores, loadings, and explained variance.

15.5 `modules/modeling.py`

Function/Class	Purpose
`_normalize_task_type`	Standardizes task names.
`_is_classification`	Checks whether task is classification.

`get_model_family`	Categorizes model as linear, nonlinear kernel, distance, tree ensemble, neural, or probabilistic.
`available_models_for_task`	Builds the supported model catalog.
`ModelBundle`	Stores model, preprocessing, descriptors, labels, metrics, and metadata.
`_prepare_target`	Converts regression targets to numeric or classification labels to encoded classes.
`_pick_cv`	Chooses KFold or StratifiedKFold.
`_run_cv`	Performs cross-validation and returns metric summaries.
`_safe_predict_proba`	Safely obtains class probabilities when supported.
`_decode_if_needed`	Converts encoded class labels back to original labels.
`train_and_compare_models`	Trains selected models, evaluates them, and returns a leaderboard and best model.
`summarize_model_bundle`	Creates a readable model summary.
`save_model_bundle`	Saves the complete model bundle with joblib.
`load_model_bundle`	Reloads a saved model bundle.
`predict_with_bundle`	Predicts new peptide sequences using the saved descriptor and preprocessing settings.
`compute_model_feature_importance`	Extracts model importance/coefficient summaries where available.
`compute_shap_importance`	Computes SHAP-based importance when SHAP supports the estimator.

15.6 `modules/evaluation.py`

Function	Purpose
`evaluate_regression`	Calculates R2, RMSE, and MAE.
`evaluate_classification`	Calculates accuracy, precision, recall, F1, ROC-AUC when available, and confusion matrix.

15.7 `modules/visualization.py`

Function	Purpose
`_top_variance_features`	Selects visually informative descriptor columns.
`plot_descriptor_distribution`	Plots descriptor distributions.
`plot_correlation_heatmap`	Plots descriptor correlation heatmap.
`plot_embedding`	Plots PCA/t-SNE/UMAP embeddings.
`plot_model_comparison`	Plots model leaderboard.
`plot_feature_importance`	Plots top influential descriptors.
`plot_prediction_ranking`	Plots ranked peptide predictions.
`plot_actual_vs_predicted`	Plots regression actual vs predicted values.
`plot_residuals`	Plots regression residuals.

`plot_confusion_matrix`	Plots classification confusion matrix.
`plot_pca_explained_variance`	Plots PCA explained variance.
`plot_pca_loading_bar`	Plots descriptor loadings for a PCA component.
`plot_structure_quality_distribution`	Plots structure-quality score distribution.
`plot_structure_criteria_radar`	Plots one peptide's design criteria as a radar chart.

| `plot_position_residue_heatmap` | Plots residue-position activity associations. |
| `plot_position_recommendation_scores` | Plots mutation recommendation scores. |

15.8 `modules/reporting.py`

Function	Purpose
`dataframe_to_csv_bytes`	Converts tables to downloadable CSV.
`dataframe_to_excel_bytes`	Converts one or more tables to an Excel workbook.
`_table_html`	Converts table previews to HTML.
`_metrics_html`	Converts metric dictionaries to HTML.
`_figures_html`	Embeds Plotly figures as HTML.
`generate_html_report`	Generates a timestamped HTML summary report.

15.9 `modules/structure_criteria.py`

Function	Purpose
`evaluate_structure_criteria`	Scores peptide candidates against practical design-quality criteria.
`summarize_structure_quality`	Summarizes structure-quality scores across peptides.

15.10 `modules/design_suggestions.py`

Function	Purpose
`infer_optimization_direction`	Infers whether high or low target values are preferred.
`_pearson_binary`	Calculates safe Pearson correlation for binary residue-position indicators.
`positional_activity_analysis`	Calculates position-residue effects relative to target values.
`suggest_mutations_for_sequence`	Suggests candidate substitutions for a selected peptide.

15.11 `launch_streamlit.py` and `peptide_qsar_single_file.py`

| Function | Purpose |

| --- | --- |
`launch_streamlit.main`	Starts Streamlit from a Windows-friendly launcher.
`peptide_qsar_single_file.runtime_root`	Chooses the local runtime extraction folder.
`peptide_qsar_single_file.materialize_project`	Writes embedded project files to the runtime folder.
`peptide_qsar_single_file.main`	Opens the browser and launches the embedded Streamlit app.

---

16. RECOMMENDED MANUSCRIPT METHODS TEXT

The following paragraph may be adapted in a scientific manuscript:

> Peptide sequences were processed using a custom Python-based Peptide QSAR Prediction Tool. The tool validates standard one-letter amino acid sequences and converts each peptide into sequence-derived descriptors, including amino acid composition, optional dipeptide composition, sequence length, molecular weight, aromaticity, Kyte-Doolittle hydropathy/GRAVY, estimated net charge at pH 7.0, isoelectric point, instability index, aliphatic index, Boman index, Shannon entropy, residue-class frequencies, elemental composition, motif frequencies, and secondary-structure propensity descriptors. Physicochemical descriptors were calculated using Biopython ProtParam where available and custom implementations for peptide-specific indices. Descriptor matrices were preprocessed by imputation, scaling, low-variance filtering, and optional correlation filtering. Regression or classification models were trained using scikit-learn estimators, with optional XGBoost/LightGBM support. Model performance was evaluated using held-out test metrics and cross-validation. Regression models were assessed using R2, RMSE, and MAE, whereas classification models were assessed using accuracy, precision, recall, F1 score, ROC-AUC where applicable, and confusion matrices. Model interpretation was performed using model-derived feature importance and optional SHAP analysis. Candidate peptides were ranked by predicted activity/property and additionally screened using sequence-derived peptide design-quality criteria.

---

17. RECOMMENDED REPORTING CHECKLIST

For a publication or thesis, report:

- Number of peptides used.
- Peptide source/database or experimental source.
- Target property and units.
- Whether the target was transformed, for example log(IC50).
- Input file format.
- Descriptor groups used.

- Whether DPC descriptors were enabled.
- Preprocessing settings.
- Train/test split ratio and random seed.
- Cross-validation folds.
- Models compared.
- Final selected model.
- Regression/classification metrics.
- Feature importance or SHAP interpretation.
- Any peptide design suggestions treated as computational hypotheses.
- Experimental validation plan for top candidates.

---

18. LIMITATIONS AND BEST PRACTICES

1. The tool is only as reliable as the dataset quality.
2. Small peptide datasets can overfit easily, especially with DPC descriptors.
3. Random split performance may be optimistic for highly similar peptides.
4. Sequence similarity or scaffold-based splitting should be considered for rigorous validation.
5. Docking scores are computational proxies and should not be treated as experimental activity.
6. Position-level suggestions are association-based and require validation.
7. Negative R2 indicates poor generalization rather than a software error.
8. For IC50/MIC-like targets, log transformation is often scientifically preferable.
9. For classification, class imbalance should be inspected before interpreting accuracy.
10. All top candidates should be checked experimentally and, when needed, by docking, molecular dynamics, toxicity prediction, and synthesis feasibility.

---

19. KEY DOCUMENTATION AND SCIENTIFIC REFERENCES

Software documentation

- Streamlit documentation: https://docs.streamlit.io/
- Streamlit `st.file_uploader`: https://docs.streamlit.io/develop/api-reference/widgets/st.file_uploader
- Streamlit `st.plotly_chart`: https://docs.streamlit.io/develop/api-reference/charts/st.plotly_chart
- Biopython ProtParam: https://biopython.org/wiki/ProtParam
- scikit-learn user guide: https://scikit-learn.org/stable/user_guide.html
- scikit-learn cross-validation documentation: https://scikit-learn.org/stable/modules/cross_validation.html
- pandas documentation: https://pandas.pydata.org/docs/
- Plotly Python documentation: https://plotly.com/python/

- SHAP documentation: https://shap.readthedocs.io/
- PyInstaller documentation: https://pyinstaller.org/en/stable/

Scientific references

- Kyte, J. and Doolittle, R. F. (1982). A simple method for displaying the hydropathic character of a protein. Journal of Molecular Biology. DOI: https://doi.org/10.1016/0022-2836(82)90515-0
- Ikai, A. (1980). Thermostability and aliphatic index of globular proteins. Journal of Biochemistry. PubMed: https://pubmed.ncbi.nlm.nih.gov/7462208/
- Guruprasad, K., Reddy, B. V. B., and Pandit, M. W. (1990). Correlation between stability of a protein and its dipeptide composition: a novel approach for predicting in vivo stability of a protein from its primary sequence. Protein Engineering. DOI: https://doi.org/10.1093/protein/4.2.155
- Chou, P. Y. and Fasman, G. D. (1978). Prediction of the secondary structure of proteins from their amino acid sequence. Advances in Enzymology and Related Areas of Molecular Biology.
- Rawlings, N., Ashman, K., and Wittmann-Liebold, B. (1983). Computerised version of the Chou and Fasman protein secondary structure predictive method. DOI: https://doi.org/10.1111/j.1399-3011.1983.tb02124.x
- Boman, H. G. (2003). Antibacterial peptides: basic facts and emerging concepts. Journal of Internal Medicine. DOI: https://doi.org/10.1046/j.1365-2796.2003.01228.x
- Lundberg, S. M. and Lee, S. I. (2017). A unified approach to interpreting model predictions. NeurIPS. SHAP documentation: https://shap.readthedocs.io/

---

20. CUSTOMIZATION NOTES

Researchers can modify:

- Descriptor formulas and scales in `modules/descriptor_engine.py` and `modules/constants.py`.
- Accepted sequence and target column names in `modules/io_utils.py`.
- Model choices and hyperparameters in `modules/modeling.py`.
- Design-quality rules in `modules/structure_criteria.py`.
- Position-level suggestion logic in `modules/design_suggestions.py`.
- Visualization style in `modules/visualization.py`.

For modified residues or non-natural amino acids, add:

- New allowed residue codes.
- Molecular weight/atom constants.
- Hydrophobicity or physicochemical scales.
- Clear reporting that descriptors were extended beyond standard amino acids.